

Plano — Etapa de Protótipo: de mockup estático a protótipo React navegável

O problema que este plano resolve

Hoje a etapa "Interface & Protótipo" produz, por tela, uma página HTML estática com Tailwind por CDN e uma imagem renderizada. Serve para olhar. Não serve para **usar**: não navega, não tem estado, não tem dado nenhum, e não é o mesmo código que vira o aplicativo depois.

Essa distância entre o protótipo e o aplicativo gerado foi exatamente o que produziu o pior episódio deste projeto: a especificação de interface declarava painel com três indicadores e dois gráficos, o mockup mostrava isso, e o aplicativo nasceu com um título e um botão — porque o emissor de código descartava calado os componentes que não sabia desenhar. Ninguém viu, porque o que se aprovava (a imagem) e o que se entregava (o código) eram artefatos distintos.

Princípio

O protótipo é o aplicativo com a fonte de dados trocada. O mesmo emissor de componentes, os mesmos nomes de campo, as mesmas ações. No protótipo, os dados vêm de um provedor fictício; no aplicativo, do servidor de agentes e do banco. Nada mais muda.

Consequência imediata: se um componente não aparece no protótipo, ele também não vai aparecer no aplicativo — e isso fica visível na hora de aprovar a etapa, não três etapas adiante.

O que a etapa passa a entregar

1. **Projeto React completo e navegável** — todas as telas, o menu, as rotas, os estados de carregando/vazio/erro, gerado a partir da especificação de interface aprovada.
2. **Dados fictícios coerentes** — derivados do Modelo de Dados aprovado: tipos, domínios de enumeração, chaves estrangeiras que casam entre as tabelas, e o vocabulário do caso de uso (um caso classificado como "ICSAC Confirmado", um alerta de multirresistência, um paciente). Uma semente por projeto, versionada e editável, para o protótipo ser sempre o mesmo.
3. **Protótipo rodando dentro da própria etapa** — a página da etapa embute o protótipo e o usuário navega ali mesmo, sem instalar nada.

4. **Refino conversando com o agente sobre a tela aberta** — o usuário aponta a tela, escreve o que quer mudar, e o agente altera a especificação de interface; o protótipo recarrega com a mudança aplicada.
5. **Contrato de tela** — a versão aprovada vira referência: a Geração de Código compara o que emitiu com o que foi aprovado e acusa toda diferença, em vez de descartar em silêncio.

Como se liga ao que já existe

Peça existente	Papel no protótipo
Emissor de componentes do gerador de código	passa a ser compartilhado: uma fonte de dados fictícia, outra real
Especificação de interface (telas, componentes, ações)	continua sendo a fonte; ganha os dados de exemplo
Modelo de Dados aprovado	origem dos dados fictícios (tipos, enumerações, relacionamentos)
Chat de refino da etapa	passa a receber também a tela aberta e o componente apontado
Vocabulário do caso de uso	rótulos, mensagens e estados que o protótipo exibe

Fases

Fase 1 — Protótipo navegável. Emitir o projeto React a partir da especificação de interface, reutilizando o emissor de componentes do gerador. Entrega verificável: abrir o protótipo, percorrer todas as telas pelo menu, ver cada componente declarado desenhado.

Fase 2 — Dados fictícios do Modelo de Dados. Gerar a semente de dados a partir do schema aprovado, com integridade entre tabelas, e um provedor que responde às ações da tela. Entrega verificável: o painel mostra números, a tabela mostra linhas, o formulário salva e a lista reflete.

Fase 3 — Protótipo dentro da etapa. Servir e embutir o protótipo na página, com seletor de tela e de versão. Entrega verificável: aprovar ou refinar sem sair da etapa.

Fase 4 — Refino apontando a tela. Enviar ao agente a tela aberta, o componente apontado e a instrução em linguagem natural; aplicar na especificação de interface e recarregar. Entrega verificável: pedir "troque o campo X por uma seleção com estes valores" e ver mudar.

Fase 5 — Contrato de tela no portão. A Geração de Código compara o emitido com o protótipo aprovado, componente a componente, e reprova a diferença. Entrega verificável: remover à mão um componente do emissor e ver o portão acusar.

Riscos e como tratá-los

- **Dado fictício virar dado de verdade no aplicativo.** É o erro que já cometemos com o "-35%" da especificação. Mitigação: a semente vive num arquivo separado, nunca entra no pacote do aplicativo, e o portão da Geração de Código reprova se algum valor da semente aparecer no código gerado.
- **Protótipo divergir do aplicativo com o tempo.** Mitigação: emissor único. Se o protótipo e o aplicativo puderem ser gerados por caminhos diferentes, a divergência volta.
- **Peso da etapa.** Um projeto React por versão ocupa espaço. Mitigação: guardar só a especificação e a semente; o projeto é reconstruído sob demanda.